A MINIMAL FIELD HANDBOOK

The Minimal Hugging Face Handbook

Models, datasets, fine-tuning, LoRA and generation - the pretrained-ML working set.

Mosafer

transformers · datasets · PEFT · accelerate

V1.0 - COMPACT EDITION

NOVEMBER 2026

Contents

1 Orientation **2**
- 1.1 The ecosystem map . . . 2
- 1.2 Thirty seconds to a prediction . . . 2
- 1.3 When to reach for HF, and when not to . . . 2

2 Models and Tokenizers **4**
- 2.1 Tokenizers . . . 4
- 2.2 Model heads . . . 4
- 2.3 Saving, dtype, and placement . . . 5

3 Datasets **6**
- 3.1 Loading . . . 6
- 3.2 Transforming . . . 6
- 3.3 Into a DataLoader . . . 6
- 3.4 Datasets too big for disk . . . 7

4 Fine-Tuning with Trainer **8**
- 4.1 The full script . . . 8
- 4.2 The arguments that matter . . . 9
- 4.3 Metrics and resuming . . . 9

5 A Custom Training Loop **10**
- 5.1 The loop, assembled . . . 10
- 5.2 HF conveniences worth keeping . . . 11

6 Generation **12**
- 6.1 The knobs . . . 12
- 6.2 Chat templates . . . 12

7 PEFT: LoRA **14**
- 7.1 The mechanism in one paragraph . . . 14
- 7.2 The recipe . . . 14
- 7.3 After training . . . 15

8 Accelerate and Quantization **16**
- 8.1 Accelerate: one script, any hardware . . . 16
- 8.2 Quantization: shrink the weights . . . 16

A Cheat Sheet **18**
- A.1 Load and save . . . 18
- A.2 Datasets . . . 18
- A.3 Trainer . . . 18
- A.4 Generation and LoRA . . . 19
- A.5 Idioms worth memorizing . . . 19

CHAPTER 1

Orientation

The Hugging Face ecosystem is the package manager of modern NLP (and much of multimodal ML): pretrained models, datasets, and training utilities behind one API convention—`from_pretrained`. This handbook covers the working minimum: load and inspect models, stream datasets, fine-tune with `Trainer` or a raw PyTorch loop, generate text, and scale with LoRA and quantization.

1.1 The ecosystem map

Package	Import as	For
transformers	`transformers`	models, tokenizers, `Trainer`, `generate`
datasets	`datasets`	dataset hub + fast local processing
accelerate	`accelerate`	device placement, multi-GPU, mixed precision
peft	`peft`	LoRA and other parameter-efficient methods
bitsandbytes	`bitsandbytes`	4/8-bit quantization
tokenizers	(inside transformers)	fast Rust tokenizers

```
pip install transformers datasets accelerate peft bitsandbytes
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

1.2 Thirty seconds to a prediction

```
from transformers import pipeline

clf = pipeline("sentiment-analysis")
clf("The Minimal Hugging Face Handbook is a joy to read.")
# [{'label': 'POSITIVE', 'score': 0.9998}]
```

The `pipeline` picks a default checkpoint, downloads it to `\textasciitilde /.cache/huggingface/` (about 250 MB for a small BERT), wraps tokenization and inference, and returns labels. Pipelines are for trying things; everything past Chapter 2 is about what happens underneath.

1.3 When to reach for HF, and when not to

Reach for HF	Stay in raw PyTorch (Ch. 4–9 of the PyTorch handbook)
Starting from a pretrained checkpoint	novel architecture research
Standard tasks: classify, generate, translate	custom training dynamics (RL, GANs)
Quick fine-tunes and experiments	learning how training really works
Multi-GPU and quantized inference	full control over the step

THE ONE CONVENTION

Everything is `AutoModel*.from_pretrained(name)` and `AutoTokenizer.from_pretrained(name)` where `name` is a Hub id (`distilbert/distilbert-base-uncased`) or a local path. Tokenizer and model must always come from the *same* name—this one rule prevents the most common HF bug.

PITFALL

Checkpoint names moved: many models gained an organization prefix (`bert-base-uncased` \(\to\) `google-bert/bert-base-uncased`). The old short names still resolve today, but pin the full id in serious projects—a default model changing under your experiment is a reproducibility bug (Ch. 11 of the PyTorch handbook).

CHAPTER 2

Models and Tokenizers

A text model is two files: a *tokenizer* (text \to integer ids) and a *model* (ids \to predictions). The `Auto*` classes load both from the same checkpoint name and give you the right subclass automatically.

2.1 Tokenizers

```
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
enc = tok("Minimal handbooks, maximal signal.", truncation=True,
          max_length=16)
enc["input_ids"]    # [101, 14831, 17724, ..., 102]
enc["attention_mask"]  # [1, 1, 1] - 1 = real token, 0 = padding
tok.convert_ids_to_tokens(enc["input_ids"])
# ['[CLS]', 'minimal', 'handbooks', ',', 'maximal', 'signal', '.', '[SEP]']
```

Things to notice: text arrives lowercase (this checkpoint is uncased); words split into subword pieces (`tokenizers` may become `token ####izer`); and the sequence is wrapped in special tokens. Batched calls pad to the longest sequence and return masks:

```
batch = tok(["short one", "a much longer sentence here"],
            padding=True, truncation=True, return_tensors="pt")
batch["input_ids"].shape      # (2, 7) - torch tensors, ready for a model
```

> **PITFALL**
>
> `padding=True` pads to the longest sample *in the batch*, which changes every batch — fine for training with masks, wrong for generation-time caching. Decoder models (GPT-style) want `tok.pad_token = tok.eos_token` and `padding_side="left"` for batched generation; the `pad_token_id` warning at generation time is this exact issue.

2.2 Model heads

`AutoModel` returns the bare backbone; the `AutoModelFor*` classes add task heads:

Class	Output
`AutoModel`	hidden states — no head
`AutoModelForSequenceClassification`	logits per class (`num_labels`)
`AutoModelForCausalLM`	next-token logits over the vocab
`AutoModelForTokenClassification`	per-token labels (NER)
`AutoModelForSeq2SeqLM`	encoder–decoder generation (T5)

```python
from transformers import AutoModelForSequenceClassification
import torch

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert/distilbert-base-uncased", num_labels=2,
    torch_dtype=torch.float32)
model.config.id2label = {0: "NEGATIVE", 1: "POSITIVE"}

with torch.no_grad():
    logits = model(**batch).logits      # (B, 2)
preds = logits.argmax(dim=-1)
```

The head is randomly initialized at load (the famous "newly initialized weights" warning)—that is expected: you fine-tune it.

2.3 Saving, dtype, and placement

```python
model.save_pretrained("out/my-model")     # config + weights (safetensors)
tok.save_pretrained("out/my-model")
model = AutoModelForSequenceClassification.from_pretrained("out/my-model")

# inference on modest GPUs: load in half precision or let accelerate shard
big = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-1B", torch_dtype=torch.bfloat16,
    device_map="auto")                    # requires accelerate
print(sum(p.numel() for p in big.parameters()) / 1e6, "M params")
```

`torch_dtype=torch.bfloat16` halves memory and speeds up inference on any Ampere-or-newer GPU (bf16 needs no gradient scaler, unlike fp16). `device_map="auto"` shards layers over the GPUs and CPU you have (Ch. 8).

WHERE DOWNLOADS LIVE

Everything fetched from the Hub is cached under `\textasciitilde /.cache/huggingface/` — models in `hub/`, datasets in `datasets/`. A crashed download leaves a corrupt cache entry; `from_pretrained(..., force_download=True)` or deleting that model's folder fixes it. Set `HF_HUB_OFFLINE=1` to run experiments from cache only.

CHAPTER 3

Datasets

The datasets library does two jobs: it hosts tens of thousands of ready datasets, and it processes data *out-of-core*—memory-mapped Arrow files, so datasets larger than RAM are fine.

3.1 Loading

```
from datasets import load_dataset

ds = load_dataset("imdb")            # {"train": 25k, "test": 25k}
ds["train"][0]                       # {"text": "...", "label": 0}
ds["train"].features                 # Columns and types, incl. ClassLabel
```

`load_dataset` accepts a Hub id, local files (`"csv"`, `"json"`, `"parquet"` builders), or a directory. Local CSV/JSONL is the everyday case:

```
ds = load_dataset("csv", data_files={"train": "train.csv",
                                     "test": "test.csv"})
```

3.2 Transforming

Processing is functional: each call returns a new dataset, and `map` runs your function in batched, parallel, cached fashion:

```
def tokenize_fn(batch):
    return tok(batch["text"], truncation=True, max_length=256)

enc = ds.map(tokenize_fn, batched=True, remove_columns=["text"])
enc = enc.filter(lambda ex: len(ex["input_ids"]) > 4)
train = enc["train"].shuffle(seed=0).select(range(2000))
```

- `batched=True` passes *lists* of examples — orders of magnitude faster than per-example.
- `remove_columns` drops raw text once tokenized; keeping it wastes memory and breaks default collators.
- Results are *cached* by function fingerprint: change the function body and it reruns; force with `load_from_cache_file=False` or `keep_in_memory=True`.
- `.set_format("torch")` makes `ds[i]` return torch tensors — handy for hand-rolled loops.

3.3 Into a DataLoader

The bridge to the Chapter 5 (PyTorch handbook) loop is a *data collator*—it pads each batch to the batch's longest sequence and builds the masks:

```
from torch.utils.data import DataLoader
from transformers import DataCollatorWithPadding
```

```
collate = DataCollatorWithPadding(tokenizer=tok)
loader = DataLoader(enc["train"].shuffle(seed=0), batch_size=32,
                    collate_fn=collate, num_workers=2, shuffle=True)
for batch in loader:
    # batch: {"input_ids", "attention_mask", "labels"} - all padded to fit
    ...
```

Collator	For
`DataCollatorWithPadding`	classification / token tasks
`DataCollatorForLanguageModeling`	MLM pretraining (masks tokens)
`DataCollatorForSeq2Seq`	encoder–decoder with label padding

3.4 Datasets too big for disk

```
stream = load_dataset("wikitext", "wikitext-103-raw-v1",
                      split="train", streaming=True)
for ex in stream.take(1000):        # no download-at-once, iterate lazily
    ...
```

Streaming iterates over the network (or a shard list) with no local copy—slower per epoch, unbounded corpus size. Shuffling a stream requires a buffer: `stream.shuffle(buffer_size=10_000, seed=0)`.

PITFALL

Two silent data bugs: (1) forgetting `truncation=True` in the tokenize function — one long document then explodes the whole batch (or errors on the model's 512-position limit); (2) splitting *after* deduplication versus before — near-duplicate reviews spanning train and test inflate your metric. Split first, dedupe inside splits, and keep the split seed in the config.

CHAPTER 4

Fine-Tuning with Trainer

`Trainer` is the Chapter 6 (PyTorch handbook) loop, productized: it owns the epoch loop, mixed precision, logging, checkpointing, and evaluation, and it is configured entirely through `TrainingArguments`. This chapter is the complete worked example — sentiment classification on IMDB with DistilBERT.

4.1 The full script

```
from datasets import load_dataset
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer,
                          DataCollatorWithPadding)
import evaluate, numpy as np

MODEL = "distilbert/distilbert-base-uncased"
tok = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL, num_labels=2)

ds = load_dataset("imdb")
enc = ds.map(lambda b: tok(b["text"], truncation=True, max_length=256),
             batched=True, remove_columns=["text"])

acc = evaluate.load("accuracy")
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    return {"accuracy": (logits.argmax(-1) == labels).mean()}

args = TrainingArguments(
    output_dir="out/imdb",
    per_device_train_batch_size=32,
    per_device_eval_batch_size=64,
    learning_rate=2e-5,
    num_train_epochs=2,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    fp16=True,                        # bf16=True on Ampere or newer
    logging_steps=100,
    report_to="none",                 # or "tensorboard", "wandb"
)

trainer = Trainer(model=model, args=args,
                  train_dataset=enc["train"].shuffle(seed=0).select(range=4000),
                  eval_dataset=enc["test"].select(range=1000),
                  data_collator=DataCollatorWithPadding(tokenizer=tok),
                  compute_metrics=compute_metrics)

trainer.train()
metrics = trainer.evaluate()
trainer.save_model("out/imdb/best")      # model + tokenizer + config
```

```
tok.save_pretrained("out/imdb/best")
```

That is genuinely the whole program: roughly forty lines for a fine-tuned classifier, with checkpoints, metrics, and mixed precision handled. Fine-tuning LRs are 10x smaller than from-scratch ones (`2e-5` here) and two epochs usually suffice—more than that erodes the pretrained features.

4.2 The arguments that matter

Argument	Meaning
`learning_rate`	`2e-5`–`5e-5` for encoder fine-tunes; linear warmup then decay by default
`eval_strategy`	`"epoch"`, `"steps"` (with `eval_steps`), or `"no"`
`save_strategy` + `load_best_model_at_end`	keep checkpoints, reload the best by eval metric
`fp16` / `bf16`	mixed precision; bf16 preferred on Ampere+ (no scaler needed)
`gradient_accumulation_steps`	simulate bigger batches (Ch. 6, PyTorch handbook)
`metric_for_best_model`	defaults to `"eval_loss"`; set to `"accuracy"` with `greater_is_better=True`
`report_to`	`"tensorboard"`, `"wandb"`, `"none"`

4.3 Metrics and resuming

`compute_metrics` receives `(logits, labels)` after each evaluation; the evaluate library loads standard metric implementations (`accuracy`, `f1`, `rouge`). Return a dict — every key becomes `eval_<key>` in the logs and the comparison table.

```
trainer.train(resume_from_checkpoint=True)   # continues the latest ckpt
preds = trainer.predict(enc["test"].select(range=500))
preds.predictions.shape, preds.metrics       # logits + timing/loss
```

PITFALL

The three classic Trainer failures: (1) `KeyError: 'labels'` — your dataset has `label` (singular); `Trainer` matches column names to the model's forward signature, so rename with `enc = enc.rename_column("label", "labels")`; (2) OOM with `pad_to_multiple_of=8` absent — dynamic padding is on, so a long outlier batch dies late in training; cap with `max_length` in tokenization; (3) `load_best_model_at_end=True` without matching `save_strategy` — the two strategies must be the same kind (both `"epoch"` or both `"steps"`).

WHEN TO GRADUATE FROM TRAINER

`Trainer` covers supervised fine-tuning end to end. The moment the loss mixes objectives (RL, adversarial, custom schedules with per-group LRs), write the Chapter 5 loop instead — Trainer's escape hatches (`compute_loss` overrides) get ugly faster than plain PyTorch gets clean.

CHAPTER 5

A Custom Training Loop

Trainer hides the loop; sooner or later you need it back—custom losses, adversarial objectives, RL fine-tunes. The bridge is small, because HF pieces are ordinary PyTorch objects: the model is an `nn.Module`, the collator emits tensors, and the Chapter 6 loop of the PyTorch handbook drops straight in.

5.1 The loop, assembled

```
import torch, torch.nn as nn
from torch.utils.data import DataLoader
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          DataCollatorWithPadding, get_scheduler)

MODEL = "distilbert/distilbert-base-uncased"
device = "cuda" if torch.cuda.is_available() else "cpu"
tok = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL, num_labels=2).to(device)

enc = ds.map(lambda b: tok(b["text"], truncation=True, max_length=256),
             batched=True, remove_columns=["text"])
loader = DataLoader(enc["train"].shuffle(seed=0), batch_size=32,
                    collate_fn=DataCollatorWithPadding(tok), shuffle=True)

opt = torch.optim.AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
sched = get_scheduler("linear", optimizer=opt,
                      num_warmup_steps=100,
                      num_training_steps=2 * len(loader))

for epoch in range(2):
    model.train()
    for step, batch in enumerate(loader):
        batch = {k: v.to(device) for k, v in batch.items()}
        logits = model(**batch).logits            # keys = forward args
        loss = nn.functional.cross_entropy(logits, batch["labels"])

        opt.zero_grad(set_to_none=True)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
        sched.step()                              # HF schedule: per step
        if step % 100 == 0:
            print(f"ep {epoch} step {step} loss {loss.item():.4f}")
```

The subtle line is `model(**batch)`: the collator produces exactly the keyword arguments the forward pass declares (`input_ids`, `attention_mask`, `labels`), and HF models compute the task loss *inside* forward when `labels` is present — `model(**batch).loss` is an alternative to the explicit cross-entropy above.

5.2 HF conveniences worth keeping

- `get_scheduler` — the linear-warmup-then-decay recipe every fine-tune uses; step it per *batch* (it is a per-step scheduler, unlike epoch-level `CosineAnnealingLR`).
- `model.save_pretrained(...)` / `tok.save_pretrained(...)` after training — reloadable with `from_pretrained` like any Hub checkpoint.
- `accelerate.prepare(model, loader, opt)` — one call to place everything on device, wrap DDP, and enable mixed precision (Ch. 8); the loop above stays unchanged.

PITFALL

Two loop regressions specific to HF models: (1) calling `model(batch["input_ids"])` and forgetting `attention_mask` — the model then attends to padding tokens and metrics degrade mysteriously; always unpack with `model(**batch)`. (2) putting the model in `model.train()` and never back — dropout stays on at evaluation; wrap validation in `model.eval()` with `torch.no_grad()`, exactly as in the PyTorch handbook.

SHARING TO THE HUB

`model.push_to_hub("mosafer/my-model", token=hf_token)` uploads weights, config, and card scaffolding; `huggingface_cli login` stores the token once. The Hub id is then a load path like any local directory — same `from_pretrained`, now versioned and shareable.

CHAPTER 6

Generation

Decoders (GPT, Llama, Qwen, Mistral) produce text token by token: predict a distribution over the vocabulary, sample one, append, repeat. `model.generate()` is that loop with every interesting knob — this chapter is the knob guide and the chat-template convention.

6.1 The knobs

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

name = "HuggingFaceTB/SmolLM2-360M-Instruct"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name).to("cuda")

prompt = "Explain LoRA in one sentence."
inputs = tok(prompt, return_tensors="pt").to("cuda")

out = model.generate(
    **inputs,
    max_new_tokens=64,          # NEW tokens (not total length!)
    do_sample=True,             # False = greedy: deterministic, repetitive
    temperature=0.7,            # <1 sharper, >1 flatter distribution
    top_k=50,                   # keep only the 50 most likely tokens
    top_p=0.9,                  # nucleus: smallest set with mass >= 0.9
    repetition_penalty=1.1,     # damp already-seen tokens
    pad_token_id=tok.eos_token_id,
)
print(tok.decode(out[0], skip_special_tokens=True))
```

Setting	Behavior
`do_sample=False`	greedy argmax — safe, dull, loops on long outputs
`temperature=0.7`	standard creative setting; `0.2` for factual extraction
`top_p=0.9` + `top_k=50`	truncates the tail — the default quality combo
`num_beams=4`	beam search — better for translation/summar ization, slower

> **PITFALL**
>
> `max_length` is the TOTAL sequence (prompt included); `max_new_tokens` counts only what the model adds — use the latter, or long prompts silently truncate your budget. And on modern models, `do_sample=True` with defaults beats greedy for everything except extraction tasks: greedy generation on sampled-model outputs is where repetition loops come from.

6.2 Chat templates

Instruct models expect their conversation wrapped in a very specific markup. Hand-writing it is the classic silent failure — the model answers fine but slightly worse, and nothing errors. The tokenizer knows the format:

```
messages = [
    {"role": "system", "content": "Answer concisely."},
    {"role": "user", "content": "Why is LoRA cheaper than full fine-tuning?"},
    {"role": "assistant", "content": "Only low-rank matrices train."},
    {"role": "user", "content": "Go on."},
]
text = tok.apply_chat_template(messages, tokenize=False,
                               add_generation_prompt=True)
inputs = tok(text, return_tensors="pt").to("cuda")
out = model.generate(**inputs, max_new_tokens=128,
                     do_sample=True, temperature=0.7,
                     pad_token_id=tok.eos_token_id)
print(tok.decode(out[0][inputs["input_ids"].shape[1]:],
                 skip_special_tokens=True))
```

`add_generation_prompt=True` appends the assistant-turn header so the model continues the conversation; slicing the output at `input_ids.shape[1]` returns only the newly generated part.

CHOOSING A CHECKPOINT

`*Instruct` / `*Chat` suffixed models are fine-tuned for the chat-template format — use the template with them. Base models continue text; they do not answer questions unless prompted as continuation tasks. For CPU or small GPUs, the `SmolLM2`, `Qwen2.5-0.5B/1.5B`, and `Llama-3.2-1B` class runs comfortably in bf16.

PITFALL

Batched generation requires `tok.padding_side = "left"` and a real `pad_token` (`tok.pad_token = tok.eos_token` for Llama-style tokenizers) — with right padding the continuation positions land in the middle of padded rows and outputs scramble. The warning `A decoder-only architecture is being used, but right-padding was detected` is this exact bug firing before the scramble.

CHAPTER 7

PEFT: LoRA

Full fine-tuning updates every weight: a 7B model means 7B gradients, 7B optimizer states, checkpoints in tens of gigabytes. LoRA (Low-Rank Adaptation) freezes the backbone and learns small rank-r matrices added to attention (and MLP) weights—typically under 1% of the parameters, at near-full quality.

7.1 The mechanism in one paragraph

For a weight matrix $W \in \mathbb{R}^{d\times k}$, LoRA learns $\Delta W = BA$ where $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$, and $r \in \{4, 8, 16, 32\}$ is tiny. The forward pass becomes $Wx + BAx$; A starts near zero (so the model starts unchanged), B starts at zero, and only these matrices plus a few norms receive gradients. Memory drops because optimizer states (Adam keeps two moments *per trainable parameter*) shrink from 7×7B to a few million.

7.2 The recipe

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model, TaskType
import torch

name = "HuggingFaceTB/SmolLM2-360M-Instruct"
model = AutoModelForCausalLM.from_pretrained(name, torch_dtype=torch.bfloat16)
tok = AutoTokenizer.from_pretrained(name)
tok.pad_token = tok.eos_token

lora = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,                       # rank: capacity of the update
    lora_alpha=32,              # scaling: effectively alpha / r on the update
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
)
model = get_peft_model(model, lora)
model.print_trainable_parameters()
# trainable params: 2.4M || all params: 360M || trainable%: 0.67

# then: the Chapter 4 Trainer or Chapter 5 loop, unchanged
# trainer = Trainer(model=model, args=args, ...); trainer.train()
```

`target_modules` names the linear layers to adapt; the string `"all-linear"` is a reasonable default for causal LMs. The `alpha/r` ratio sets the update's effective scale—doubling both keeps the model equivalent; the `r=16, alpha=32` pair is the common starting point.

7.3 After training

```
model.save_pretrained("out/lora-smollm")      # ~10-50 MB: only the adapters

# reload later:
from peft import PeftModel
base = AutoModelForCausalLM.from_pretrained(name, torch_dtype=torch.bfloat16)
model = PeftModel.from_pretrained(base, "out/lora-smollm")

# or fold the adapters into the weights for plain inference:
model = model.merge_and_unload()              # now a plain CausalLM
model.save_pretrained("out/merged")
```

Adapter files are tiny, which changes the economics: one 7B backbone, dozens of task adapters, swapped at load time.

PITFALL

Three LoRA classics: (1) `get_peft_model` called *twice*—wrapping an already-wrapped model silently misconfigures target modules; (2) saving with `model.save_pretrained` before training (adapters are random near-zero—the saved file is useless) versus after; (3) gradient checkpointing off—with LoRA the `batch_size` is usually GPU-bound, so pair it with `model.gradient_checkpointing_enable()` and `model.enable_input_requirement_grads()` when memory is tight.

QLORA: FINE-TUNING ON ONE CONSUMER GPU

QLoRA = load the base model in 4-bit (bitsandbytes, Ch. 8) + LoRA on top. A 7B model fine-tunes in under 10 GB:
`BitsAndBytesConfig(load_in_4bit=True)`.
then the identical `LoraConfig` recipe. Quality is within noise of 16-bit LoRA for instruction tuning — this is the default single-GPU fine-tuning recipe of 2026.

CHAPTER 8

Accelerate and Quantization

Two tools scale what you already have: accelerate moves a single script across devices (one GPU, multi-GPU, mixed precision) with minimal edits, and bitsandbytes shrinks models so they fit wherever you want them.

8.1 Accelerate: one script, any hardware

```
from accelerate import Accelerator

accel = Accelerator()               # reads the environment
model, opt, loader, sched = accel.prepare(model, opt, loader, sched)

for batch in loader:
    with accel.accumulate(model):   # handles gradient accumulation
        loss = step_loss_fn(model, batch)
        accel.backward(loss)        # instead of loss.backward()
        opt.step()
        sched.step()
        opt.zero_grad(set_to_none=True)
```

`accel.prepare` places tensors on the right device, wraps the model in DDP when multiple GPUs are visible, and activates mixed precision per the launch flags:

```
accelerate config           # one-time questionnaire, writes a config
accelerate launch train.py  # single or multi-GPU, per the config
accelerate launch --num_processes=4 train.py
```

The loop above is the Chapter 5 loop with three edits (`accel.backward`, `accumulate`, `prepare`)—and it now runs on four GPUs unchanged. `Trainer` uses accelerate internally, which is why its multi-GPU story is just `torchrun`/`accelerate launch`.

> **PITFALL**
>
> Inside an accelerate run, do not call `model.to(device)` or hardcode `"cuda:0"` — placement belongs to `prepare`. Save with `accel.unwrap_model(model).save_pretrained(...)` from the main process only (`accel.is_main_process`), or you store `module.`-prefixed DDP weights (the same trap as Ch. 12 of the PyTorch handbook).

8.2 Quantization: shrink the weights

```
from transformers import BitsAndBytesConfig, AutoModelForCausalLM
import torch

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",        # best quality for LLM weights
```

```
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-3B-Instruct",
    quantization_config=bnb, device_map={"": 0})
print(model.get_memory_footprint() / 1e9, "GB")
```

Precision	3B model footprint	Use
float32	~12 GB	training from scratch; rarely needed
bfloat16	~6 GB	the default for training and inference
4-bit NF4	~2 GB	inference and QLoRA on small GPUs

Quantization compresses the *weights*; activations and KV-cache still cost memory during generation. Combined with LoRA (Ch. 7), 4-bit loading is the QLoRA recipe.

A SCALING LADDER

A useful ladder for fine-tuning on one machine: bf16 full fine-tune → bf16 + LoRA → 4-bit QLoRA + gradient checkpointing → accumulate or shard with `device_map="auto"`. Each rung trades a little speed or quality for a lot of memory; QLoRA + checkpointing fits most 7–8B fine-tunes into a 24 GB card.

PITFALL

`load_in_8bit=True` and `bnb_4bit_quant_type="nf4"` are mutually exclusive settings of the same config—pick one. And quantized models train only through PEFT adapters: calling `model.parameters()` for a full `AdamW` on a 4-bit model either errors or silently freezes—quantized weights are not updated by gradients.

APPENDIX A

Cheat Sheet

A.1 Load and save

Task	Code
Load	`AutoModelForX.from_pretrained(name)`, `AutoTokenizer.from_pretrained(name)`
Save	`model.save_pretrained(dir)`, `tok.save_pretrained(dir)`
Half precision	`from_pretrained(..., torch_dtype=torch.bfloat16)`
Shard	`from_pretrained(..., device_map="auto")`
Quantize	`BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4")`
Tokenize	`tok(texts, padding=True, truncation=True, return_tensors="pt")`
Decode	`tok.decode(ids, skip_special_tokens=True)`

A.2 Datasets

Task	Code
Load	`load_dataset("imdb")` / `("csv", data_files=...)`.
Transform	`ds.map(fn, batched=True, remove_columns=[...])`
Filter/select	`ds.filter(fn)`, `ds.shuffle(seed)`, `ds.select(range(n))`
Torch tensors	`ds.set_format("torch")`
Stream	`load_dataset(..., streaming=True)`, `.take(n)`
Collate	`DataCollatorWithPadding(tok)` / `DataCollatorForLanguageModeling`

A.3 Trainer

Task	Code
Core args	`output_dir`, `learning_rate=2e-5`, `num_train_epochs`, `per_device_train_batch_size`
Eval/save	`eval_strategy="epoch"`, `save_strategy="epoch"`, `load_best_model_at_end=True`
Speed	`bf16=True`, `gradient_accumulation_steps`
Metrics	`compute_metrics=lambda p: <"accuracy": ...\>`
Run	`trainer.train()` / `trainer.train(resume_from_checkpoint=True)` / `trainer.evaluate()`

A.4 Generation and LoRA

Task	Code
Generate	`model.generate(**inputs, max_new_tokens=128, do_sample=True, temperature=0.7, top_p=0.9)`
Chat format	`tok.apply_chat_template(messages, add_generation_prompt=True)`
Batched gen	`tok.padding_side = "left"`; `tok.pad_token = tok.eos_token`
LoRA	`LoraConfig(r=16, lora_alpha=32, target_modules="all-linear")` + `get_peft_model(model, cfg)`
Save adapters	`model.save_pretrained(dir)` (~10–50 MB)
Merge	`model.merge_and_unload()`
QLoRA	4-bit base + LoRA; adapters train, weights stay frozen

A.5 Idioms worth memorizing

```
tok = AutoTokenizer.from_pretrained(name)      # same name as the model!
enc = tok(texts, padding=True, truncation=True, return_tensors="pt")
batch = {k: v.to(device) for k, v in batch.items()}
out = model(**batch).logits                    # keys = forward args
text = tok.apply_chat_template(messages, tokenize=False,
                               add_generation_prompt=True)
model.gradient_checkpointing_enable()          # memory for activations
accel.prepare(model, opt, loader)              # device/DDP/mp in one call
```

THE HABITS THAT MATTER

Tokenizer and model from the same checkpoint name; truncation always on; labels column named `labels`; fine-tune with `2e-5` and two epochs; generate with `max_new_tokens` and left padding; save adapters, not 7B copies. Everything else is in the docs — and now you know what to search for.